

Program	B. Tech	Semester- 5
Type of Course	Open Elective	
Prerequisite	C language, Data Structure, Discrete Mathematics	
Course Objective	To understand the basics concept of Graph theory and implement a relative Programs.	

Teaching Scheme				Examination Scheme				
Lecture	Tutorial	Lab	Credit	Theory Marks		Practical Marks		Total Marks
				SEE	CIA	SEE	CIA	
3	0	2	4	40	30	20	10	100

1	Objective: This practical is designed as a revision of graph concepts previously studied in Discrete Mathematics and Data Structures. Problem 1: Given the number of vertices and edges of a graph, the task is to represent the adjacency list of a directed graph. Input: $V=3$, edges $[[0,1],[1,2],[2,0]]$ Output: 0 -> 1 1 -> 2 2 -> 0 Input: $V=4$, edges $[[0,1],[1,2],[1,3],[2,3],[3,0]]$ Output: 0 -> 1 1 -> 2 3 2 -> 3 3 -> 0 Problem 2: Write a program to perform the following basic graph-related operations: 1. Represent a graph using adjacency matrix. 2. Represent a graph using adjacency list. 3. Find the degree of each vertex in the graph. 4. Determine whether the graph is simple, complete, or connected. 5. Display total number of vertices and edges. Input: $V = 5$, edges $[[0, 1), (0,2), (1,2), (2,3), (3,4)]$ Output: 1. Adjacency Matrix 2. Adjacency List 3. Degree of each vertex 4. Graph properties 5. Total vertices and edges
2	Objective: To implement Breadth First Search traversal for a graph. Problem: Perform a Breadth First Search (BFS) traversal starting from vertex 0. Input: $adj[[1,2], [0,2,3], [0,1,4], [1,4], [2,3]]$ Output: [0, 1, 2, 3, 4]
3	Objective: To implement Depth First Search traversal for a graph. Problem: Write a program to perform DFS traversal starting from a given source vertex. Input: $adj[[1,2], [0,2], [0,1,3,4], [2], [2]]$ Output: [0, 1, 2, 3, 4]

4	Objective: To understand the basic properties and structure of trees using graph operations. Problem 1: Write a program to perform the following operations on a tree: 1. Identify all pendent (leaf) vertices. 2. Find the degree of each vertex. 3. Find distance between vertices. 4. Find eccentricity of each vertex 5. Find the center of the tree. 6. Verify whether the given graph is a tree or not. Input: $V = 6$, edges $E = \{(0,1), (1,2), (1,3), (3,4), (3,5)\}$ Output: 1. List of pendent vertices 2. Degree of each vertex 3. Distance matrix 4. Eccentricity of each vertex 5. Center of the tree 6. Confirmation whether the graph is a tree
5	Objective: To analyze the structural properties of a connected graph using spanning trees, branches, chords, rank, and nullity. Problem 1: In a social network of N people, some of them are directly connected as friends. If person A is a friend of person B, and person B is a friend of person C, then A, B, and C are all part of the same friend circle (a connected component in graph terms). You are given an undirected graph represented by an adjacency matrix of size $N \times N$, where $matrix[i][j] = 1$ indicates a direct friendship between person i and person j. Your task is to determine the total number of friend circles. Input: <pre>4 // total N people 1 1 0 0 // adj. Matrix 1 1 0 0 0 0 1 1 0 0 1 1</pre> Output: <pre>2</pre> Problem 2: Write a program to perform the following operations on a connected graph: 1. Construct a spanning tree of the graph. 2. Identify all branches and chords. 3. Find all fundamental circuits 4. Calculate: Rank and Nullity of the graph 5. Verify that: Number of branches = Rank and Number of chords = Nullity. Input: <pre>Enter number of vertices: 5 Enter number of edges: 6 edges[E] = {(0,1),(0,2),(1,2),(1,3),(2,4),(3,4)}</pre> Output: <pre>Branches: (0,1) (0,2) (1,3) (2,4) Chords: (1,2) (3,4) Fundamental Circuits: C1 = {(0,1),(0,2),(1,2)} C2 = {(1,3),(2,4),(3,4)} Rank of Graph = 4 Nullity of Graph = 2</pre>

6	Objective: To identify articulation points in an undirected graph and determine connectivity and separability of a graph. Problem: Write a program to find all cut-vertices of a graph. Input: Connected graph with vertices and edges. Output: List of articulation points.
7	Objective: Determine connectivity and separability of a graph Problem: Write a program to: 1. Find vertex connectivity 2. Find edge connectivity 3. Construct fundamental cut-sets 4. Determine whether graph is separable or non-separable Input: Undirected graph. Output: Connectivity values and graph type
8	Objective: To determine the dominating set and domination number of a graph. Problem: Write a program to: 1. Find dominating sets 2. Find domination number 3. Find total dominating sets 4. Find total domination number Input: $N = 4, M = 4$, Edges = $\{(1,2), (1,3), (3,4), (2,4)\}$ Output: Possible Dominating Sets: $\{1,4\}$ $\{2,3\}$ $\{1,2\}$ $\{1,3\}$ $\{2,4\}$ $\{3,4\}$ Domination Number = 2 Total Dominating Sets: $\{1,3\} \{2,4\}$ Total Domination Number = 2
9	Objective: To determine the total dominating set and total domination number of a graph. Problem: Write a program to: 1. Find total dominating sets 2. Find total domination number Input: $N = 4, M = 4$, Edges = $\{(1,2), (1,3), (3,4), (2,4)\}$ Output: Total Dominating Sets: $\{1,3\} \{2,4\}$ Total Domination Number = 2
10	Objective: To determine different types of matching in graphs Problem: Write a program to determine: Maximal matching, Maximum matching, and Perfect matching Input: Bipartite graph. Output: Matching sets and type of matching.
11	Objective: Verify Hall's Condition Problem: Write a program to check whether Hall's condition holds for a given bipartite graph. Input: Bipartite graph with sets X and Y. Output: Hall's condition satisfied or not satisfied.

12	Objective: To construct different matrices associated with graphs. Problem: Write a program to construct: Adjacency Matrix, Degree Matrix, and Laplacian Matrix Input: $N = 5$, $M = 4$, Edges = $\{(1,2), (2,3), (4,5), (1,5)\}$ Output Adjacency, Degree, and Laplacian matrices.
13	Objective: To compute eigenvalues and eigenvectors of graph matrices. Problem: Write a program to calculate eigenvalues and eigenvectors of adjacency and Laplacian matrix. Input: Adjacency or Laplacian matrix. Output: Eigenvalues and corresponding eigenvectors.
14	Objective: To perform spectral clustering on a graph using the Laplacian matrix and eigenvectors. Problem: Write a program to perform spectral clustering on an undirected graph by using the graph Laplacian matrix. The program should: 1. Construct the adjacency matrix of the graph. 2. Compute the degree matrix. 3. Construct the Laplacian matrix. 4. Compute eigenvalues and eigenvectors of the Laplacian matrix. 5. Partition the graph into clusters using spectral properties. Input: $N = 6$ Edges = $\{(0,1),(0,2),(1,2),(3,4),(4,5),(3,5)\}$ Output: Cluster 1: $\{0,1,2\}$ Cluster 2: $\{3,4,5\}$
15	Objective: To find shortest paths between all pairs of vertices. Problem: Write a program to implement Floyd–Warshall Algorithm. Input: Weighted graph represented using adjacency matrix. Output: Shortest path matrix between all pairs of vertices.